

Theatre Audience Forecasting

1. Project Overview

This notebook builds a regression pipeline to forecast movie theatre audience counts (daily footfall per theatre) using historical booking, visit, and theatre-metadata records. The problem is framed as a supervised regression task where the target variable is the daily `audience_count` for a given theatre and show date.

Two theatre-booking ecosystems are combined as data sources — a legacy point-of-sale platform (“`cinePOS`”) and a newer booking platform (“`booknow`”) — each with its own theatre and booking records, unified through a theatre ID mapping table.

1.1 Data Sources

File	Description	Rows × Cols
<code>movie_theatre_id_relation.csv</code>	Maps <code>cinePOS</code> theatre IDs to <code>booknow</code> theatre IDs	150 mapped IDs
<code>date_info.csv</code>	Calendar reference with day-of-week label	547×2
<code>booknow_theatres.csv</code>	Theatre metadata (type, area, lat/long) — <code>booknow</code> platform	— (515 missing lat/long)
<code>cinePOS_theatres.csv</code>	Theatre metadata — <code>cinePOS</code> platform	$4,690 \times 5$
<code>cinePOS_booking.csv</code>	Individual booking transactions — <code>cinePOS</code>	—
<code>booknow_booking.csv</code>	Individual booking transactions — <code>booknow</code>	$68,336 \times 4$
<code>booknow_visits.csv</code>	Ground-truth daily audience counts (target)	$214,046 \times 3$

2. Data Cleaning & Preprocessing

2.1 Missing Value Audit

A null-value check was run across every source table. Two notable gaps were found:

- `booknow_theatres`: 515 missing `book_theatre_id` entries.
- `cinePOS_theatres`: 3,861 of 4,690 rows missing latitude/longitude (only 829 populated) — confirmed via `.info()`.

All other fields (booking timestamps, ticket counts, day-of-week, audience counts) were fully populated with zero nulls.

2.2 Outlier Detection & Treatment (IQR Method)

A helper function computed the interquartile range (Q1, Q3) per numeric column and flagged values beyond $1.5 \times \text{IQR}$ from the quartiles. Detected outlier rates:

Dataset / Column	Outliers Found	% of Rows
booknow_visits — audience_count	5,589	2.61%
cinePOS_booking — tickets_sold	100,686	6.13%
booknow_booking — tickets_booked	5,542	8.11%
cinePOS_theatres — latitude	77	9.29%
cinePOS_theatres — longitude	0	0%

Rather than dropping these rows, outliers were winsorized — capped at the lower/upper IQR bounds — preserving sample size while limiting the influence of extreme booking spikes on downstream aggregation and modeling.

2.3 Unifying the Two Booking Platforms

cinePOS and booknow booking tables were harmonized onto a common schema (theatre_id, show_dt, booking_dt, tickets) and concatenated into a single all_booking table, with theatre IDs cross-walked via the mapping table so records from both platforms could be aggregated together.

Timestamps were coerced to datetime, invalid rows dropped, and a booking_lag (hours/days between booking time and show time) was derived. Records with a negative lag (bookings logged after the show) were removed as invalid.

2.4 Aggregation to Theatre-Day Booking Features

The cleaned transaction-level booking data was grouped by (theatre_id, show_date) to produce theatre-day level features:

- total_booked — sum of tickets booked for that theatre/date
- mean_booked — average tickets per booking transaction
- median_lag_days — median booking lead time
- booking_count — number of distinct booking transactions

These were merged onto the target table (booknow_visits) along with calendar info (date_info) and theatre metadata (type, area, latitude, longitude). Remaining nulls after the merge were imputed: booking-derived features filled with 0 (no bookings recorded), lat/long filled with the column mean, and categorical metadata filled with the label “Missing”.

3. Exploratory Data Analysis

3.1 Audience Count Distribution

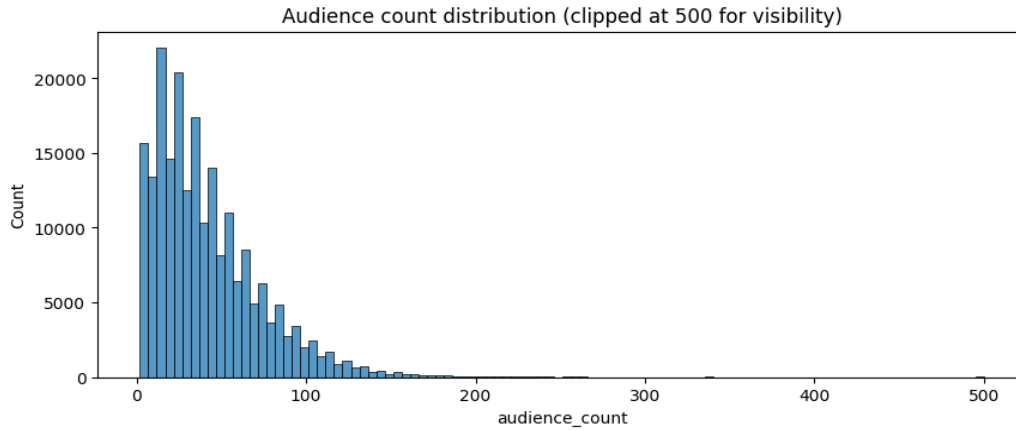


Figure 1 — Distribution of daily `audience_count` (clipped at 500 for visibility). The distribution is heavily right-skewed, with most theatre-days drawing a small-to-moderate audience and a long tail of high-attendance outliers.

3.2 Aggregate Audience Over Time

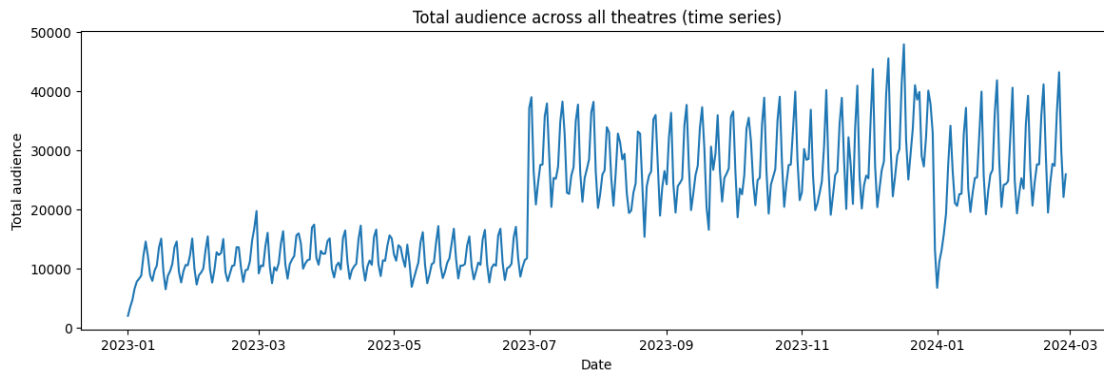


Figure 2 — Total audience summed across all theatres per show date, revealing clear temporal fluctuation and periodic spikes consistent with weekly/seasonal demand cycles.

3.3 Audience by Theatre Type

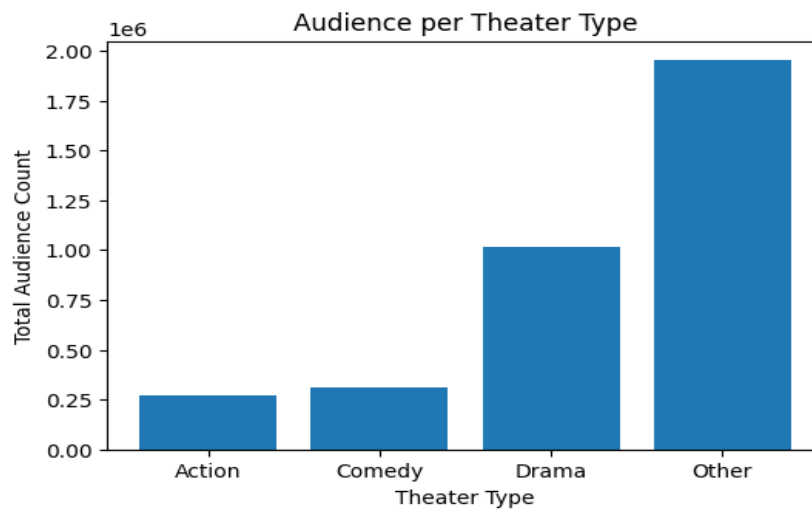


Figure 3 — Total audience aggregated by theatre_type. “Other” and “Comedy” categories dominate theatre counts (411 and 196 theatres respectively out of 829 typed booknow theatres), which is reflected in cumulative audience volume.

3.4 Ticket Booking Spread

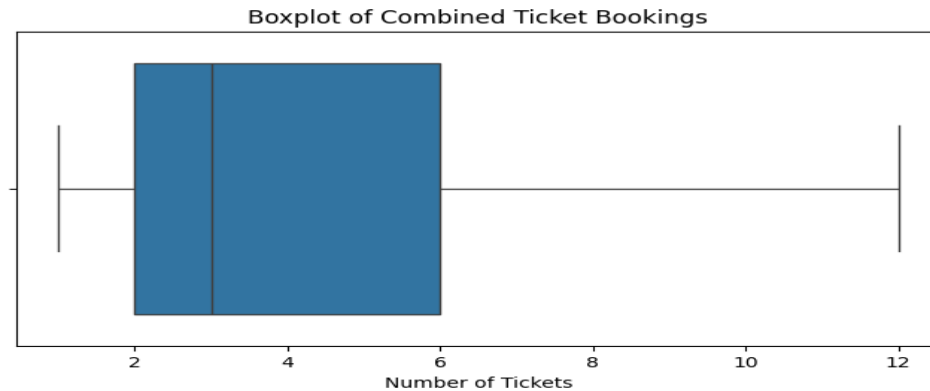


Figure 4 — Boxplot of the combined (cinePOS + booknow) ticket-booking counts, illustrating the concentration of typical bookings and the extended right tail later addressed via IQR capping.

3.5 Other Notable EDA Findings

- 214,046 daily theatre-visit records span 424 unique show dates and 547 calendar reference days.
- cinePOS_booking contains 13,161 unique cine_theatre_id values, while cinePOS_theatres lists 4,690 theatres — indicating not every registered theatre has booking activity in the sample.
- The theatre ID mapping table links 150 unique book_theatre_id values between the two platforms.
- theatre_type value counts (booknow_theatres): Other (411), Comedy (196), Drama (183), Action (39).

4. Feature Engineering

Beyond the booking aggregates described above, the notebook derives calendar and theatre-behavior signals to help the models capture seasonality and per-theatre baseline demand:

4.1 Calendar Features

- year, month, day, and week_of_year extracted from show_date.
- is_weekend flag derived from day_of_week (or from weekday index as a fallback).

4.2 Theatre-Level Target Statistics

- theatre_mean — historical average audience_count per theatre (target encoding at theatre level).
- theatre_dow_mean — average audience_count per theatre, per day-of-week, falling back to the overall theatre mean when a specific day-of-week combination is unseen.

Together with the booking aggregates (total_booked, mean_booked, median_lag_days, booking_count) and theatre metadata (type, area, latitude, longitude), the final modeling table combines 17 feature columns describing calendar timing, booking behavior, and theatre identity/location.

5. Modeling Preprocessing Pipeline

A scikit-learn ColumnTransformer standardizes the numeric block and one-hot encodes categoricals, wrapped inside a Pipeline together with each regressor so that preprocessing is fit only on training folds:

Feature Group	Columns	Transform
Numerical	latitude, longitude, total_booked, mean_booked, median_lag_days, booking_count, year, month, day, week_of_year, theatre_mean, theatre_dow_mean	StandardScaler
Binary	is_weekend	Passthrough
Categorical	theatre_type, theatre_area, day_of_week	OneHotEncoder (unknown categories ignored)

The dataset was split into training and test sets; the reconstructed test set size used for evaluation is 42,700 rows across 17 features.

6. Modeling & Results

Four regression approaches were trained and compared, all evaluated on the same held-out test split using Mean Absolute Error (MAE) and R^2 :

6.1 Baseline Random Forest Regressor

Configuration: 400 trees, max_depth=5, min_samples_split=10, min_samples_leaf=60 — deliberately shallow/regularized to limit overfitting on the theatre-day aggregates.

6.2 XGBoost Regressor

Configuration: 50 estimators, learning_rate=0.05, max_depth=4, squared-error objective, histogram tree method.

6.3 LightGBM Regressor

Configuration: 50 estimators, learning_rate=0.05, max_depth=3, num_leaves=50. Training log confirms 17,564 training rows and 75 encoded features after one-hot expansion.

6.4 Hyperparameter-Tuned Random Forest

A RandomizedSearchCV (50 iterations, 3-fold CV, scoring = negative MSE) was run over n_estimators, max_depth, min_samples_split, min_samples_leaf, and max_features. The best configuration found was:

- n_estimators = 420, max_depth = 20, min_samples_split = 5, min_samples_leaf = 2, max_features = 'sqrt'
- Best CV RMSE: 18.83 | Test MAE at this configuration: 14.99

Note: several candidate parameter combinations in the search grid were invalid for the installed scikit-learn version (e.g. max_features='auto', min_samples_split=1), causing those specific fits to fail and return NaN scores; RandomizedSearchCV skipped these safely and still converged on a valid best estimator.

6.5 Model Comparison

Model	MAE (test)	R^2 (test)
Random Forest (baseline)	13.50	0.606

Model	MAE (test)	R ² (test)
XGBoost	14.10	0.586
LightGBM	13.89	0.595
Random Forest (tuned via RandomizedSearchCV)	14.99	0.542

The baseline, lightly-regularized Random Forest achieved the best generalization on the test split (highest $R^2 \approx 0.61$, lowest MAE ≈ 13.5), narrowly ahead of LightGBM. Interestingly, the tuned Random Forest — optimized for cross-validated RMSE — underperformed the untuned baseline on the held-out test set, suggesting the deeper trees selected by the search (`max_depth=20`) may have overfit relative to the shallower, more regularized baseline configuration (`max_depth=5`, `min_samples_leaf=60`).

7. Summary & Observations

- Two independent booking platforms (cinePOS legacy POS, booknow modern booking) were successfully unified via an ID-mapping table into a single feature-engineering pipeline.
- IQR-based capping was used throughout to control the influence of extreme booking/audience values rather than discarding data.
- Engineered theatre-level and day-of-week target statistics (`theatre_mean`, `theatre_dow_mean`) alongside booking-lag aggregates were the core predictive signals feeding all four models.
- Tree-ensemble models (Random Forest, XGBoost, LightGBM) were benchmarked; a shallow, well-regularized Random Forest generalized best on this dataset, outperforming both gradient-boosted alternatives and a deeper hyperparameter-tuned Random Forest.
- Opportunities for improvement: revisit the `RandomizedSearchCV` parameter grid to remove invalid values, add cross-theatre lag/rolling-window features, and evaluate ensembling/stacking the top-performing models.